

School Management System (SMS) — Master Project Document

Purpose of this document: This is your single source of truth for the project — modules, versions, tech stack, SaaS conversion plan, and payment handling. Come back to this whenever you forget scope, a module detail, or a decision you made and why.

Target scale: 5,000+ users per school (Management, Staff, Students, Parents) **Long-term goal:** Start as a single-tenant system, later convert into a multi-tenant SaaS product sold to multiple schools on a monthly subscription.

1. Final Tech Stack

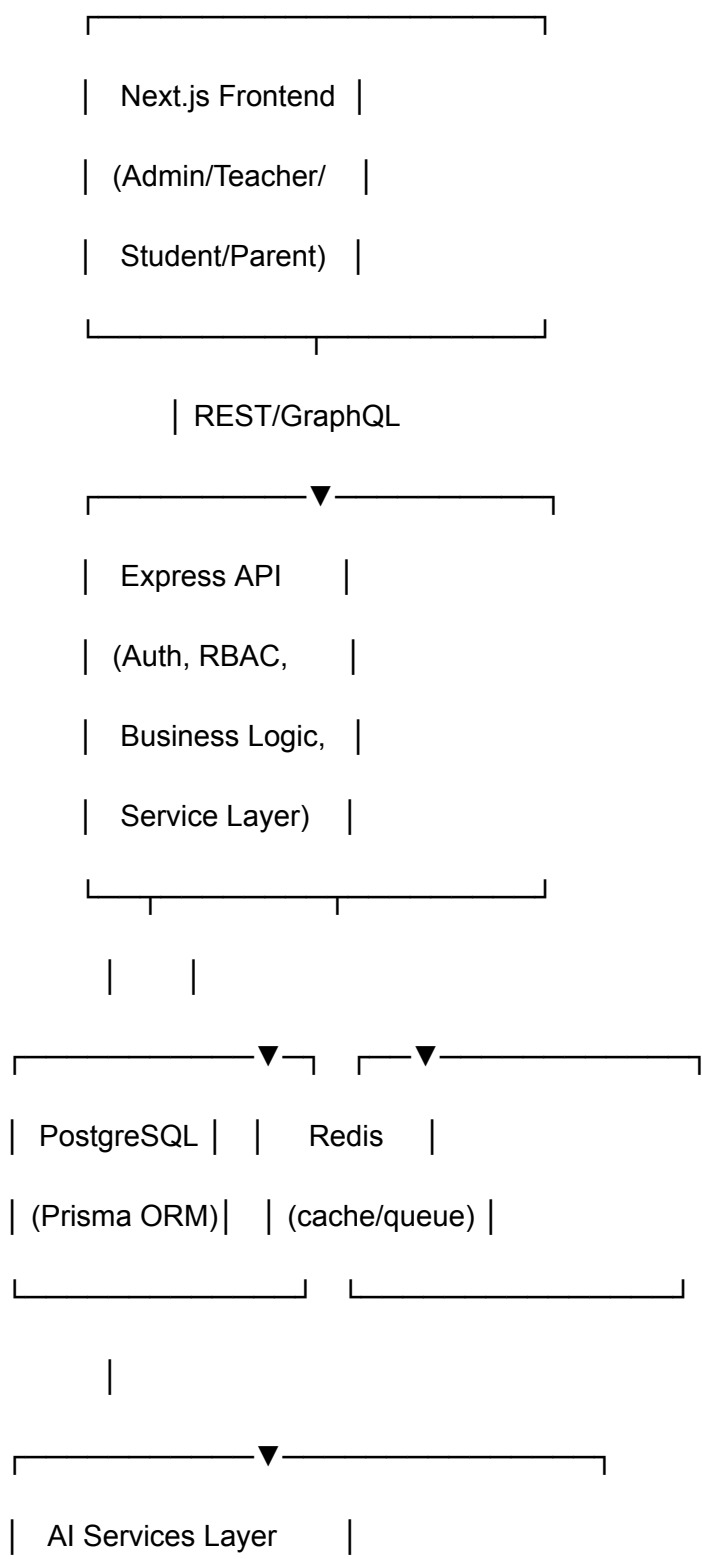
Layer	Technology	Why
Frontend	Next.js 14+ (App Router), TypeScript	SSR/SSG for dashboards, API routes, good SEO for public pages, matches existing skillset
UI	Tailwind CSS + shadcn/ui	Fast, consistent, professional UI
Data fetching	TanStack Query (React Query)	Caching, background refetch, optimistic updates
Backend	Node.js + Express.js, TypeScript	Matches MERN background, huge ecosystem
Database	PostgreSQL	Relational integrity for students/classes/fees/exams — this data is joins-heavy
ORM	Prisma	Type-safe queries, migrations, clean TS integration

Layer	Technology	Why
Cache/Queue	Redis + BullMQ	Session cache, rate limiting, background jobs (reminders, report generation)
Auth	JWT (access + refresh tokens), bcrypt	Standard, secure, scalable
File Storage	AWS S3 / Cloudinary	Documents, ID cards, profile photos
Realtime	Socket.io	Chat, live notifications
Notifications	Nodemailer (email), Firebase Cloud Messaging (push), Twilio (SMS, optional)	Multi-channel comms
AI Layer	LangChain.js + OpenAI/Claude API (Node) + optional Python microservice (FastAPI) for ML prediction	Stay in TS ecosystem; use Python only where ML tooling genuinely needs it
Vector DB	pgvector (Postgres extension)	Avoids adding a separate vector DB service for the RAG chatbot
Deployment	Docker, Nginx, AWS (EC2/RDS/S3) or Railway/Render for early hosting	Containerized, portable
CI/CD	GitHub Actions	Automated testing/deployment

Why Postgres over MySQL: relational integrity for joins-heavy data, **pgvector** support for the AI chatbot, strong JSONB support for flexible fields like custom grading rubrics.

Golden rule for the whole codebase: All database access goes through service/repository functions (e.g., `studentService.getAll()`), never raw Prisma calls scattered in route handlers. This one habit is what makes the future multi-tenancy conversion take weeks instead of months. See Section 5.

2. Architecture Overview



- | - LangChain.js (RAG bot) |
 - | - pgvector (embeddings) |
 - | - Python ML microservice |
 - | (performance prediction)|
-

Roles: Super Admin (you, the SaaS owner) → School Admin → Principal → Teacher → Student/Parent → Accountant/Librarian/Transport Manager.

3. Core Modules (Reference List)

A. User & Access Management

- Multi-role auth (JWT access + refresh tokens)
- RBAC permission matrix per module
- Password reset via email OTP
- Admin-created accounts (no public signup for students/staff)

B. Student Information System (SIS)

- Admission workflow: enquiry → application → document upload → approval → enrollment
- Student profile: personal info, guardian info, medical info, previous school records
- Class/section assignment, promotion/transfer between sessions
- ID card generation (PDF + QR code)
- Bulk import via CSV/Excel

C. Staff Management

- Onboarding, document verification, role assignment
- Payroll basics (salary structure, deductions, payslips)
- Leave management (apply, approve/reject, balance tracking)
- Performance/appraisal (optional advanced feature)

D. Academic Management

- Class, Section, Subject, Academic Session setup
- Timetable (manual builder + conflict detection; auto-generation later)

- Assignments/homework (upload, submit, grade, feedback)
- Exam management: creation, seating plan, marks entry, grade calculation, report cards
- Configurable grading systems (percentage, GPA, custom rubric)

E. Attendance Management

- Daily attendance per class
- Biometric/RFID/QR integration hooks (mockable initially)
- Leave requests from students/parents
- Attendance analytics (defaulter list, monthly %, parent alerts)

F. Fee & Finance Management

- Fee structure setup per class/category (tuition, transport, hostel, exam)
- Invoice generation, receipts, due reminders
- Discounts/scholarships
- Financial reports (collection, pending dues, defaulters)
- **Payment collection method — see Section 8 (Payment Handling) for current approach**

G. Communication Module

- Announcements (school-wide, class-wide, role-specific)
- Email/SMS/push notifications
- Parent-teacher messaging/chat (Socket.io)
- Event calendar

H. Library Management

- Book catalog, issue/return, fine calculation
- Reservation queue, overdue alerts

I. Transport Management

- Route/vehicle/driver management
- Student-route mapping
- Live tracking hook (mock or real GPS)
- Pickup/drop notifications

J. Hostel Management (optional)

- Room allocation, fee integration, visitor logs

K. Reports & Analytics Dashboard

- Role-specific dashboards

- Exportable reports (PDF/Excel)
- Charts: attendance trends, performance trends, fee collection

L. Document Management

- Certificate generation (bonafide, transfer certificate, character certificate)
 - Template-based dynamic data merge → PDF
-

4. AI Modules (Reference List, Priority Order)

1. **RAG Chatbot** (highest priority) — LangChain.js + OpenAI/Claude + pgvector. Grounded strictly in the school's own documents (policies, fee dates, syllabus, FAQs). Available to students/parents/staff.
2. **Student Performance Prediction** — Python (FastAPI + scikit-learn) microservice. Inputs: attendance %, submission rate, marks trend. Output: risk score (Low/Medium/High) surfaced to teachers/admin.
3. **Automated Report Card Comments** — LLM-generated remarks from marks/attendance data, always reviewed/edited by teacher before finalizing (human-in-the-loop). Generated via background job (BullMQ), not live per-request.
4. **AI Question Paper Generator** — Teacher inputs topic/difficulty/count → LLM drafts questions + answer key → teacher approves before publishing.
5. **Assignment Similarity/Plagiarism Check** — Embedding-based cosine similarity across submissions in the same assignment; flags for teacher review, never auto-rejects.
6. **AI-Assisted Timetable Generation** (stretch) — Constraint-satisfaction/backtracking algorithm; LLM only used to explain the generated schedule in plain language.
7. **Face Recognition Attendance** (optional/stretch) — `face-api.js` (TensorFlow.js) or Python `face_recognition` library.

Rule of thumb: build 2-3 of these deeply (Chatbot + Prediction + Report Comments is the strongest combo) rather than all 7 shallowly.

5. Versioned Roadmap

V1 — Foundation (MVP)

- Auth & RBAC (JWT, password reset)
- User/Profile management (Admin CRUD for Students/Staff/Parents, bulk CSV import)
- Academic structure setup (Class, Section, Subject, Session)
- Basic attendance (mark, view, % calculation)

- Role-specific dashboard
- Announcements

● V2 — Academic Core

- Timetable (manual builder + conflict detection)
- Assignments/homework (submit, grade, feedback)
- Exam management (seating plan, marks entry, grading, report cards)
- Leave management (student + staff)
- Notification expansion (email + in-app real-time via Socket.io)

● V3 — Finance & Extended Operations

- Fee management (structure, invoices, receipts, defaulter list, reminders)
- **Payment collection: manual/bank-transfer based initially — see Section 8**
- Library management (catalog, issue/return, fines)
- Transport management (routes, mock GPS, pickup/drop alerts)
- Document generation (certificates via templates)
- Reports & analytics v1 (attendance/fee/performance charts, PDF/Excel export)

● V4 — AI Integration Phase 1

- RAG chatbot (student/parent/staff support)
- Automated report card comments (human-reviewed)
- AI question paper generator (human-reviewed)
- Assignment similarity/plagiarism check

● V5 — AI Advanced + Scale

- Student performance prediction (Python microservice, at-risk dashboard)
- AI-assisted timetable auto-generation
- Face recognition attendance (optional)
- Load testing for 5,000+ concurrent users, DB indexing/query optimization
- Redis caching for dashboard aggregates
- Rate limiting on AI endpoints
- Audit logging for sensitive admin actions

● V-SaaS — Multi-Tenancy & Subscription Conversion

(Do this once V1-V3 are stable and the product is proven with one school; do not build this on day one.)

Why this is doable later without a rewrite: as long as V1-V3 were built with all DB access behind service/repository functions (the "golden rule" in Section 1), converting to multi-tenant is a scoped 2-6 week project, not a rewrite. See the effort table below.

What changes:

1. **Schema:** add `schoolId` to every tenant-scoped table + migration/backfill script (existing data becomes "School #1")
2. **Auth:** JWT payload includes `schoolId`; login resolves which school a user belongs to
3. **Query layer:** service functions updated once to auto-filter by `schoolId` (this is why centralizing data access in V1 matters so much)
4. **Defense-in-depth:**
 - App-level: Prisma middleware auto-injects `schoolId` into every query
 - DB-level: Postgres Row-Level Security (RLS) policies as a safety net, so even a missed filter can't leak data across schools
5. **Super Admin panel:** new — onboard schools, manage subscriptions, suspend non-paying accounts
6. **Routing:** subdomain-per-school (`greenwood.yoursms.com`) or single-domain with school resolution — start with single-domain, it's simpler
7. **Subscription/billing:** `School` table gets `subscriptionPlan`, `subscriptionStatus`, billing reference fields — actual payment gateway integration comes later (Section 8)

Effort estimate (depends entirely on V1-V3 discipline):

Codebase pattern in V1-V3	Estimated time
Centralized service layer, consistent patterns	2–3 weeks
Mixed — some centralized, some scattered	4–6 weeks
Scattered queries, no service layer	6–10 weeks, higher bug/security risk

Do this now, even in single-tenant mode, to keep this cheap later:

- Add the `schoolId` column to your schema from day one, hardcoded to a single value everywhere — turns a future migration + backfill into "just start using the column you already have"
- Never call `prisma.model.method()` directly in route handlers — always go through a service function

6. Core Database Entities (Reference)

School (id, name, subdomain, subscriptionPlan, subscriptionStatus, trialEndsAt)

User (id, schoolId, email, passwordHash, role, createdAt)

Student (id, schoolId, userId, admissionNo, classId, sectionId, guardianInfo, dob)

Staff (id, schoolId, userId, designation, subjectIds[], joiningDate)

Class (id, schoolId, name)

Section (id, schoolId, classId, name)

Subject (id, schoolId, name, classId, teacherId)

Attendance (id, schoolId, studentId, date, status, markedBy)

Assignment (id, schoolId, subjectId, classId, title, dueDate, attachmentUrl)

Submission (id, schoolId, assignmentId, studentId, fileUrl, submittedAt, grade, feedback)

Exam (id, schoolId, name, classId, term, academicYear)

Mark (id, schoolId, examId, studentId, subjectId, marksObtained, maxMarks)

FeeStructure (id, schoolId, classId, category, amount, dueDate)

FeePayment (id, schoolId, studentId, feeStructureId, amountPaid, status, paymentDate, paymentMethod, referenceNote)

Book (id, schoolId, isbn, title, author, totalCopies, availableCopies)

BookIssue (id, schoolId, bookId, studentId, issueDate, dueDate, returnDate, fine)

Route (id, schoolId, name, vehicleId, driverId)

Announcement (id, schoolId, title, body, targetRole, targetClassId, createdBy)

Notification (id, schoolId, userId, message, isRead, createdAt)

AIEmbedding (id, schoolId, sourceDocId, contentChunk, embedding vector)

Note the `schoolId` column already present in every table above — this reflects the "add it now, use it later" advice from Section 5.

7. Suggested Repo Structure

sms-project/

```
├── apps/
|   ├── web/           # Next.js frontend
|   └── api/           # Express backend
|       └── src/
|           ├── routes/
|           ├── services/ # ALL DB access lives here
|           ├── middleware/
|           └── ai/
├── packages/
|   ├── db/            # Prisma schema + client
|   └── shared-types/   # Shared TS types (frontend + backend)
├── services/
|   └── ml-prediction/  # Optional Python FastAPI microservice
├── docker-compose.yml
└── README.md
```

8. Edge Cases & Real-World Scenarios (Checklist)

This section exists so you never ship a module without having at least considered the messy real-world case. Treat each bullet as a question to answer during design, not just a bug to fix later.

8.1 Onboarding & Enrollment

- Duplicate student records (same admission number re-imported, or entered twice by mistake) — enforce uniqueness at DB level, not just UI validation
- Mid-year admission — attendance % and fee proration must be calculated from actual join date, not full session start
- Student transfer between sections/classes mid-year — history must follow the student
- Student withdrawal (TC issued) — **soft delete only**; historical attendance/grades/payments must remain intact for audit/legal reasons
- Re-enrollment of a former student (alumni returning) — decide whether to reuse or archive-and-recreate their old record
- Siblings sharing one parent account, one parent linked to multiple children
- Parent with no email/phone on file — needs a fallback path (QR code on ID card for self-registration, office-assisted entry)
- A staff member who is also a parent at the same school — one login, role-switching, not two disconnected accounts

8.2 Academic Year / Session Transitions

- Year-end promotion must support exceptions (failed/repeating students), never a blanket "promote everyone"
- Historical records must stay tagged to the correct academic year, even for repeated classes
- Teacher-subject-class reassignment at new session start
- Archiving old session data while keeping it queryable (parents often want past years' report cards)

8.3 Attendance

- Half-day/partial attendance, not just present/absent
- Backdated corrections need an edit window + audit log (who changed it, when)
- Public holidays/closures excluded from attendance % — not counted as absences
- Absent teacher — substitute assignment needed before attendance can be marked
- Student transferred sections mid-week — attendance history follows the student, not the old section

8.4 Exams & Grading

- Student absent from exam recorded as "Absent," never as zero marks (changes GPA/average math)
- Re-exam/supplementary exams for a subset of students
- Grade correction after report cards are published — needs an approval workflow + audit trail, never a silent edit

- Different grading scales across sections (grades vs percentage vs GPA)
- Late marks submission from one teacher shouldn't block the whole class's report card generation — flag and proceed

8.5 Fees & Finance

- Partial payments — ledger tracks running balance, not just paid/unpaid
- Refunds on mid-term withdrawal — dedicated workflow, never just deleting the payment record
- Sibling/staff-child discounts applied via relationship data
- Fee structure changes mid-year must not retroactively affect already-invoiced students
- Overpayment creates a credit balance applied to the next invoice, not a dangling anomaly
- Late fee grace periods and partial-month proration

8.6 Auth & Access

- Separated/divorced parents needing independent (possibly restricted) access to the same child's data
- "Parent" role should really be "Guardian" with a relationship-type field (grandparent, legal guardian, etc.)
- Staff who leaves the school — **deactivate, never delete**; their historical grading/attendance entries must remain attributable
- Account recovery when the registered email/phone is also lost or outdated — needs an admin-assisted path, not just self-service reset
- Teacher-level data scoping — a teacher should only see students in *their* classes, not the whole school (relationship-based access, not just role-based)

8.7 Multi-Tenancy / SaaS

- School cancels subscription — data retained read-only for a defined grace period (e.g., 30–90 days), then deleted; document this in your terms
- School wants to migrate data from their old system during onboarding — eventually needs an import-mapping tool beyond your own CSV template
- School renaming / subdomain changes after signup
- A teacher working part-time across two different schools on your product — needs **separate accounts per school**; don't let one login span tenants, it breaks isolation

8.8 Data Privacy & Compliance

- Students are minors — strict, relationship-based access to photos/documents/contact info
- Data deletion requests vs legal retention obligations for academic records — needs a documented policy, not an ad-hoc call

- Audit logging for all sensitive edits (grades, attendance corrections, fee waivers)

8.9 Notifications

- Delivery failures need retry logic + a visible "failed" indicator to admin, never a silent drop
- Notification preferences per user (avoid fatigue — not everyone wants every alert)
- Outdated contact info silently causing notifications to go nowhere — periodic "verify your contact info" prompts help

8.10 Scale & Performance

- Predictable traffic spikes (result day, fee due-date day) — cache + rate-limit so the DB doesn't fall over
- Bulk import must be transactional — never leave a batch half-imported with no clear record of what succeeded
- Long-running report generation (full-school annual reports) runs as a background job, never blocks a request

8.11 AI-Specific

- Chatbot must have a visible "I'm not sure, contact the school office" fallback rather than confidently hallucinating
 - Performance prediction flags must be explainable ("flagged due to declining attendance + 2 missed assignments") and appealable — never a silent black-box label
 - Minimize student PII sent to third-party LLM APIs; disclose this in your privacy policy
 - Rate-limit and monitor LLM API spend per user/school to avoid cost spikes
 - Consider regional language support (e.g., Urdu) for chatbot/notifications if targeting Pakistan broadly
-

9. Payment Handling

Current approach (V3, while the product is single-tenant / early SaaS stage):

Fees collected by parents are **not** processed through an automated payment gateway yet. Instead:

- Parents pay via **bank transfer / cash / other manual method directly**, and school staff (Accountant role) record the payment manually in the system: amount, date, method, reference number/screenshot upload
- The system still generates invoices, tracks due/paid status, sends reminders, and produces receipts — it just doesn't move money itself yet

- `FeePayment.paymentMethod` field stores `manual`, and `referenceNote` stores whatever proof/reference the accountant enters
- This keeps V3 scope realistic and avoids payment-gateway compliance overhead before the product has real traction

Same approach applies to your own SaaS subscription revenue (schools paying *you* monthly): until the product is proven, schools pay your business bank account directly (bank transfer/invoice), and you manually mark their `School.subscriptionStatus` as `active/past_due` in the Super Admin panel. No gateway integration needed yet.

Future step — once the product is successful and scaling (do not build this early):

- Integrate a real payment gateway: **Stripe** (international) or **Razorpay/JazzCash/Easypaisa** (if targeting Pakistan primarily) for both:
 1. Parent → school fee payments (in-app checkout instead of manual entry)
 2. School → you subscription billing (recurring monthly charge, automatic via Stripe Billing/webhooks)
- When that time comes: listen to gateway webhooks (`invoice.paid`, `payment_failed`, `subscription.cancelled`) to auto-update `FeePayment.status` and `School.subscriptionStatus` instead of manual entry
- Pricing model to plan for later: per-student or tiered by student count (e.g., Up to 500 students / 500–2000 / 2000+) — standard in ed-tech SaaS
- This is a clean, additive change: your schema (`paymentMethod`, `subscriptionStatus` fields) is already built to support it — you're only swapping *how* the status gets updated (manual entry → automated webhook), not the data model itself

Why this order makes sense: manual payment collection is completely valid for early-stage products (many real SaaS companies operate this way before scaling), avoids gateway approval/compliance overhead too early, and because the schema already anticipates it, adding a real gateway later is a scoped feature addition, not a redesign.